



13강 실습 · 2시간

같은 숫자를, 다르게 그려보기

축 조작 · 순서.기간 조작 · 차트 유형 오용 — 9~12강 데이터를 착시 버전과 정직한 버전으로

오늘 만들 것 — 세 개의 답



축을 조작하면 얼마나 왜곡되는가, 숫자로

다크모드 데이터를 0축과 절단축 두 버전으로 그리고, lie factor(거짓 배율)를 직접 계산한다.



순서·기간을 조작하면 없는 추세가 생긴다

클라우드 사용량 데이터를 정렬 순서와 기간을 바꿔가며 그려보고, 진짜 추세와 비교한다.



차트 유형을 잘못 고르면 무엇이 왜곡되는가

파이 차트 오용과 이중축 남용을 직접 만들어보고, 정규화로 정직하게 고친다.

오늘의 최종 산출물 “같은 데이터를 착시 버전 하나, 정직한 버전 하나씩 그려보고, 그 차이를 숫자(lie factor)로 설명한다.” — 이 문장을 직접 코드로 만듭니다.

오늘의 진행 — 약 125분

8분

도입 · 오늘 쓸 데이터 4종 소개

Colab 접속, 이론 복습, 9~12강 데이터 미리보기

20분

PART 1 · 축을 조작하면 벌어지는 일

0축 vs 절단축 → lie factor 계산 → 해석

20분

PART 2 · 순서·기간을 조작하면 벌어지는 일

정렬 조작 → 실제 추세와 비교 → cherry-picking

10분

휴식

20분

PART 3 · 차트 유형을 잘못 고르면

파이차트 오용 → 이중축 남용 → 정규화로 수정

25분

PART 4 · 정직한 차트 함수 만들기

체크리스트를 함수로 강제하기 → 여러 데이터셋에 적용

15분

PART 5 · 미니 챌린지

16개 지점 절감액 데이터 — 착시·정직 버전 직접 제작

8분

정리 · 체크리스트 최종 점검 · 과제 안내

시작 전 점검 — 5분 안에 끝내기



환경 준비

- colab.research.google.com 접속 → 새 노트북
- pandas, matplotlib 모두 기본 제공
- 오늘 쓸 CSV 4개를 미리 업로드해 둘 것



13강 이론 기억 소환 (체크!)

- 막대 차트는 반드시 0에서 시작 — 안 그러면 명시
- 차트 유형은 취향이 아니라 목적(비교·추세·관계·비중)
- 정직한 시각화 = 진짜 크기만큼만 보이게 하는 것



오늘의 규칙: 착시 버전을 먼저 만들어 보고, 그다음 정직한 버전으로 고치는 순서로 진행한다.

오늘 쓸 데이터 4종 미리보기



darkmode_engagement.csv

12강 데이터 — 다크/라이트모드 사용시간 (67.6분 vs 44.6분)



cloud_usage_log.csv

10강 데이터 — 24개월 클라우드 사용시간 (월별)



incident_media_log.csv

11강 데이터 — 장애 유형별 실제 건수·내부 언급 횟수



automation_pilot_savings.csv

10강 데이터 — 16개 지점 자동화 절감액 (PART4 미니챌린지)



네 파일 모두 Colab 좌측 파일 탭에 업로드해 두세요 — 파트가 시작될 때마다 하나씩 불러옵니다.

PART 1

25분

축을 조작하면 벌어지는 일

같은 데이터를 0축과 절단축으로 각각 그려보고, 왜곡의 크기를 lie factor로 계산한다

1 0에서 시작하는 정직한 축으로 그리기

2 40부터 시작하는 절단축으로 그리기

3 lie factor(거짓 배율) 계산하기

4 해석: 축 하나 바꿨을 뿐인데

0에서 시작하는 정직한 축으로 그리기

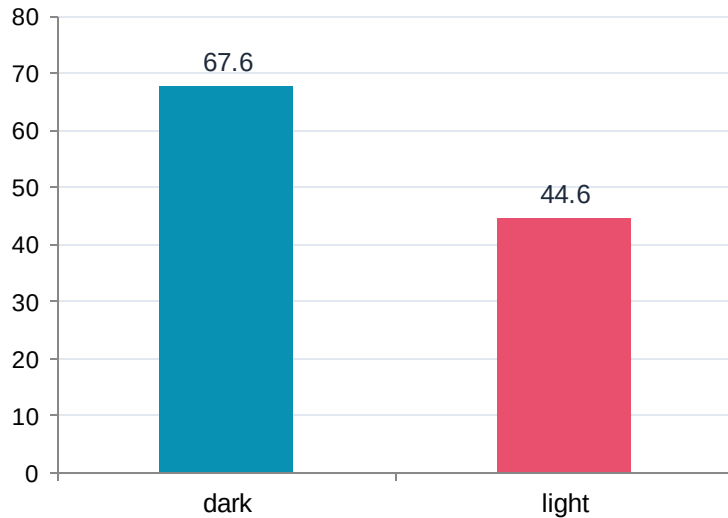


honest_axis.py

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("darkmode_engagement.csv")
m = df.groupby("theme")["daily_minutes"].mean()
print(m)

plt.bar(m.index, m.values)
plt.ylim(0, 80) # 0부터 시작
```

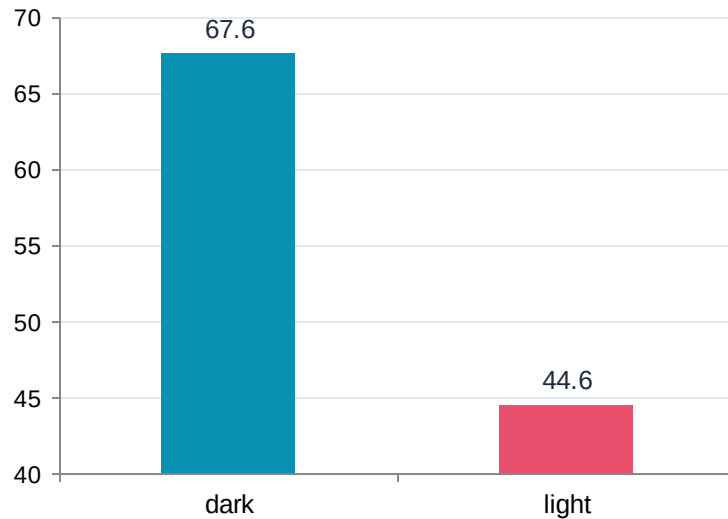


dark 67.6분, light 44.6분 — 실제 비율은 1.52배. 이 그림이 '정직한 기준'이다.

40부터 시작하는 절단축으로 그리기

```
misleading_axis.py

plt.bar(m.index, m.values,
        color=["#0891B2", "#E8506E"])
plt.ylim(40, 70) # 40부터 시작!
plt.title("다크모드가 압도적으로 좋다")
```



숫자는 STEP1과 완전히 똑같다 — 그런데 light의 막대가 거의 안 보일 만큼 작아 보인다.

lie factor(거짓 비율) 계산하기



lie_factor.py

```
actual_ratio = m["dark"] / m["light"]

base = 40
visual_ratio = (m["dark"]-base) / (m["light"]-base)

lie_factor = visual_ratio / actual_ratio
print("실제 비율:", round(actual_ratio,2))
print("시각적 비율:", round(visual_ratio,2))
print("lie factor:", round(lie_factor,2))
```

실행 결과

실제 비율: 1.52
시각적 비율: 6.04
lie factor: 3.98

약 4배 부풀려짐



통계학자 에드워드 터프티가 제안한 lie factor = 그래픽이 보여주는 효과 크기 ÷ 데이터의 실제 효과 크기. 1에 가까울수록 정직하다.

축 하나 바꿨을 뿐인데

숫자는 하나도 바꾸지 않았다

dark 67.6분, light 44.6분 — STEP1과 STEP2의 데이터는 완전히 동일했다.

그런데 인상은 완전히 달라졌다

축의 시작점을 40으로 옮겼을 뿐인데, 눈에 보이는 차이가 1.52배에서 6.04배로, lie factor 3.98배 부풀려졌다.

폭스 뉴스 ACA 차트도 원리는 똑같았다. 축의 시작점 하나가, 진짜 데이터보다 훨씬 극적인 인상을 만든다 — 그래서 막대 차트는 반드시 0에서 시작해야 한다.

내 손으로 확인하기 ① (7분)

M1

automation_pilot_savings.csv에서 site A(253만원)와 B(223만원)로 같은 lie factor를 계산해 보자

힌트: `base=150`으로 축을 잘라보면 어떻게 될까?

M2

base 값을 40 대신 60으로 바꾸면 lie factor가 어떻게 달라지는지 확인해 보자

힌트: `base`가 클수록 lie factor는 더 커질까, 작아질까? 먼저 예측 후 실행

M3

절단축 차트에 '이 그래프의 y축은 40부터 시작합니다'라는 주석을 코드로 추가해 보자

힌트: `plt.text()` 또는 `plt.annotate()` 사용

다 끝난 사람은: base를 light의 값(44.6)에 가깝게(예: 44) 만들면 lie factor가 얼마나 커지는지 확인할 것

PART 2

25분

순서·기간을 조작하면 벌어지는 일

정렬 순서와 보여주는 기간을 바꿔가며, 없는 추세가 어떻게 생기는지 확인한다

1 값 기준으로 정렬해서 그리기 (조지아주 방식)

2 실제 월별(시간순) 그래프와 비교하기

3 6개월만 잘라서 보여주기 (cherry-picking)

4 해석: 순서와 기간, 둘 다 맥락이다

값 기준으로 정렬해서 그리기

```
reorder_trick.py

cu = pd.read_csv("cloud_usage_log.csv")

sorted_desc = cu.sort_values(
    "usage_hours", ascending=False)

plt.plot(range(len(sorted_desc)),
         sorted_desc["usage_hours"])
plt.title("사용량이 계속 줄고 있다?")
```

실행 결과

1위: 190시간
2위: 153시간
3위: 145시간
⋮
24위: 23시간

190 → 23, 완벽한 하락 곡선



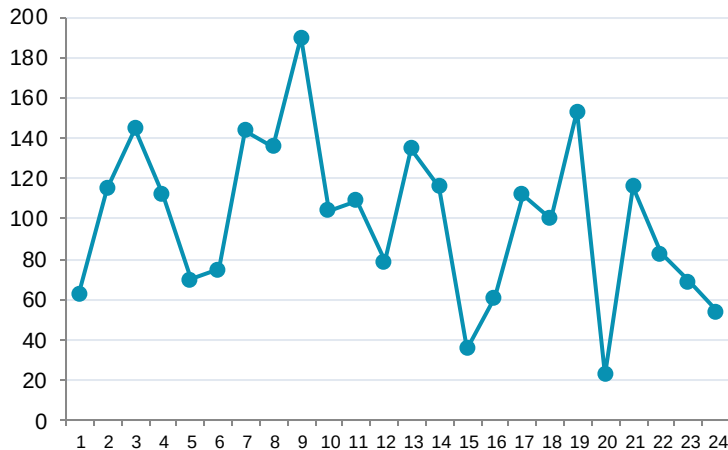
x축이 '월'이 아니라 '크기 순위'라는 것을 라벨에서 지우면, 마치 시간이 지날수록 사용량이 줄어드는 것처럼 보인다.

실제 월별(시간순) 그래프와 비교하기



true_order.py

```
plt.plot(cu["month"], cu["usage_hours"])
plt.xlabel("월")
print(cu["usage_hours"].mean())
print(cu["usage_hours"].std())
```



실제 시간순으로 그리면 평균 100.0시간 주변을 오르내릴 뿐, 어떤 추세도 없다 — STEP1의 '하락 추세'는 순서 조작이 만든 착시였다.

6개월만 잘라서 보여주기 (cherry-picking)



cherry_pick.py

```
window = cu[cu["month"].between(19, 24)]  
plt.plot(window["month"], window["usage_hours"])  
plt.title("최근 6개월, 급격한 감소세")  
  
print(window["usage_hours"].tolist())
```

실행 결과

[153, 23, 116, 83, 69, 54]

153 → 54, -99시간

19~24개월만 보면 톱 떨어진 것 같지만, 전체 24개월 표준편차(40.0)를 생각하면 그냥 원래 있던 변동폭이다.



기간을 좁게 잘라 보여주는 것도 순서를 바꾸는 것과 원리가 같다 — 맥락(전체 그림)을 지운다는 점에서.

순서와 기간, 둘 다 맥락이다

정렬 순서를 크기순으로 바꾸면 '하락 추세'가, 기간을 6개월로 좁히면 '급감'이 만들어졌다. 그러나 전체 24개월을 시간순으로 보면 평균 100.0시간 주변에서 오르내릴 뿐이다.

조지아주 보건국이 실제로 저지른 실수도 이 둘의 조합이었다. 그래프를 볼 때는 항상 '전체 범위'와 '정렬 기준'을 먼저 확인하자.



다음 파트에서는 '차트 유형' 자체를 잘못 고르는 경우를 살펴본다.

내 손으로 확인하기 ② (6분)

M1

오름차순(`ascending=True`)으로 정렬하면 이번엔 '급격한 증가세'처럼 보이는 구간을 만들 수 있는지 확인해 보자

힌트: `cu.sort_values("usage_hours", ascending=True)`

M2

가장 극적인 '증가 구간' 6개월을 직접 찾아보자 — 어느 달부터 어느 달까지가 가장 그럴듯한가

힌트: `for start in range(19):` 로 6개월 구간을 순회하며 차이를 비교

M3

cherry-picked 그래프에 '전체 24개월 평균은 100시간'이라는 안내문을 추가해 정직하게 고쳐보자

힌트: `plt.axhline(100, linestyle="--")`로 기준선을 그어본다

다 끝난 사람은: 12개월(1년) 단위로 잘라서 보여주면 결론이 어떻게 달라지는지 비교해 볼 것



10분 휴식

다음 파트에서는 차트 유형 자체를 잘못 고르는 경우 — 파이 차트와 이중축을 직접 만들어봅니다.

PART 3

25분

차트 유형을 잘못 고르면

파이 차트 오용과 이중축 남용을 직접 만들어보고, 정규화로 정직하게 고친다

- 1 파이 차트로 두 그룹 비율 표현해보기 (오용)
- 2 이중축으로 스케일이 다른 두 지표 겹쳐보기 (오용)
- 3 비율로 정규화해서 정직하게 고치기
- 4 해석: 정규화가 곧 '공정한 비교'다

파이 차트로 두 그룹 비율 표현해보기 (오용)



pie_misuse.py

```
ab = pd.read_csv("ab_test_results.csv")
rates = ab.groupby("group")["converted"].mean()
print(rates)

plt.pie(rates.values, labels=rates.index,
        autopct="%.1f%%")
```

실행 결과

control	0.096
treatment	0.127

합이 100%가 아니다!

$9.6\% + 12.7\% = 22.3\%$ — 파이 차트는 '전체 중 비중'을 보여줘야 하는데, 두 독립적인 전환율은 애초에 하나의 전체를 이루지 않는다.



matplotlib은 값을 자동으로 100%에 맞춰 다시 비율화해 그려준다 — 그래서 겉보기엔 그럴듯해 보이지만, 의미상 완전히 틀린 차트다.

이중축으로 스케일이 다른 두 지표 겹쳐보기 (오용)

```

dual_axis_misuse.py

im = pd.read_csv("incident_media_log.csv")

fig, ax1 = plt.subplots()
ax1.bar(im["incident_type"], im["actual_count_1yr"])
ax2 = ax1.twinx()  # 오른쪽에 축 하나 더!
ax2.plot(im["incident_type"],
         im["internal_mentions_count"], color="red")

```

실행 결과

실제 건수: 2~420

언급 횟수: 2~60

(스케일이 7배 이상 차이)

두 축의 눈금을 어떻게 잡느냐에 따라, 언급 횟수 선이 건수 막대와 '얼추 비슷해' 보이게 만들 수도 있다.



축이 두 개면, 두 축의 눈금 범위를 어떻게 잡느냐에 따라 사실상 원하는 인상을 마음대로 만들 수 있다.

비율로 정규화해서 정직하게 고치기

```

normalize.py

im["rate_per_100"] = (
    im["internal_mentions_count"]
    / im["actual_count_1yr"] * 100
)
print(im[["incident_type", "rate_per_100"]].
      .sort_values("rate_per_100"))

```

실행 결과

UI 버그	0.71
API 응답	1.92
소규모 장애	1.05
⋮	
보안 침해	3000.00

이제 단일 축, 단일 막대로 정직하게 비교 가능



발생 건당 언급 횟수로 정규화하니, 보안 침해 사고는 발생 1건당 언급이 UI 버그의 4,000배가 넘는다 — 이중축 없이도, 하나의 막대 차트로 명확하다.

정규화가 곧 '공정한 비교'다

파이 차트·이중축의 공통점

둘 다 '스케일이 다른 것을 억지로 하나의 그림에 욱여넣는다'는 문제였다 — 파이는 부분-전체가 아닌 것을 부분-전체처럼, 이중축은 서로 다른 두 척도를 하나의 그림처럼 보이게 한다.

해법은 같다: 정규화·분리

비율로 바꾸거나(정규화), 아예 그래프를 두 개로 나누면(패널 분리) 어느 쪽이든 단일 축·단일 척도로 정직하게 비교할 수 있다.



다음은 미니 챌린지 — 16개 지점 데이터로 착시 버전과 정직한 버전을 직접 만들어본다.

내 손으로 확인하기 ③ (7분)

M1

email_subject_test.csv의 오픈율(control 18.75% vs treatment 22.88%)로도 같은 파이 차트 오류를 만들어보자

힌트: 합이 100%가 안 되는데 파이차트가 그려지는지 직접 확인

M2

incident_media_log.csv를 이중축 대신 '막대 두 개를 나란히 놓은 서브플롯(2개 패널)'으로 다시 그려보자

힌트: plt.subplots(1, 2)로 두 그래프를 나란히

M3

rate_per_100 기준으로 정렬한 막대 그래프를 그리고, 어떤 인사이트가 보이는지 한 문장으로 써보자

힌트: 11장에서 배운 '가용성 편향'과 연결해서 생각해보기

다 끝난 사람은: 카테고리가 8개인 incident_media_log를 파이 차트로 그려보고 왜 알아보기 어려운지 확인할 것

PART 4

25분

나만의 정직한 차트 함수 만들기

오늘 배운 체크리스트를 함수 하나로 코드에 새겨, 여러 데이터셋에 재사용한다

- 1 함수가 자동으로 지켜야 할 규칙 정하기
- 2 `honest_bar()` 함수로 구현하기
- 3 세 학기 데이터셋에 적용해보기
- 4 해석: 체크리스트를 손이 아니라 코드가 지키게 하기

함수가 자동으로 지켜야 할 규칙 정하기

PART1~3에서 확인한 체크리스트를, 사람이 매번 확인하지 않아도 함수가 스스로 지키게 만들자.



규칙 1 · y축은 항상 0에서 시작

`ylim(0, ...)`을 함수 안에 고정해, 절단축을 아예 만들 수 없게 한다.



규칙 2 · 출처를 그래프에 자동 표기

`source` 인자를 받아 그래프 하단에 항상 출처 텍스트를 붙인다.



규칙 3 · 카테고리가 많으면 경고

`labels`가 5개를 넘으면 (13강 이론의 파이차트 기준과 동일) '막대 정렬을 고려하라'는 경고를 출력한다.



이 세 가지를 함수 안에 '강제'로 넣으면, 매번 체크리스트를 손으로 대조하지 않아도 된다.

honest_bar() 함수로 구현하기



honest_bar.py

```
def honest_bar(labels, values, title="", source=""):  
    if len(labels) > 5:  
        print("경고: 카테고리가 많습니다 - 정렬을 고려하세요")  
  
    fig, ax = plt.subplots()  
    ax.bar(labels, values, color="#0891B2")  
    ax.set_ylim(0, max(values) * 1.15) # 0에서 시작 강제  
    ax.set_title(title)  
    if source:  
        ax.text(1.0, -0.12, f"출처: {source}",  
               transform=ax.transAxes, ha="right", fontsize=8)  
    return fig
```



함수로 한 번 만들어두면, 이후 어떤 데이터든 이 세 가지 규칙을 어길 수 없다.

세 학기 데이터셋에 적용해보기



apply_all.py

```
ap_sorted = ap.sort_values(
    "monthly_saving_manwon", ascending=False)
honest_bar(ap_sorted["site"],
            ap_sorted["monthly_saving_manwon"],
            title="지점별 절감액",
            source="automation_pilot_savings.csv")

honest_bar(m.index, m.values,
            title="다크/라이트모드 사용시간",
            source="darkmode_engagement.csv")
```

실행 결과

- ✓ automation_pilot_savings
→ 0축, 출처 자동 표기
- ✓ darkmode_engagement
→ PART1 STEP1과
완전히 동일한 그림
- ✓ incident_media_log(8개)
→ 카테고리 경고 발생



세 데이터셋 모두 같은 함수 하나로, 축을 자르거나 출처를 빠뜨릴 걱정 없이 정직한 차트가 나온다.

체크리스트를 손이 아니라 코드가 지키게 하기

손으로 확인하면

매번 그래프를 그릴 때마다 5가지 체크리스트를 사람이 기억해서 대조해야 한다 — 바쁘면 잊어버리기 쉽다.

함수로 강제하면

`ylim(0,...)`과 출처 표기가 코드 안에 박혀 있어, 실수로라도 절단축을 쓰거나 출처를 빠뜨릴 수 없다.

10강에서 배운 자동화의 가치가 여기서도 그대로다. 좋은 습관은 '기억하기'보다 '코드로 강제하기'가 훨씬 안전하다.

내 손으로 확인하기 ④ (6분)

M1

labels가 5개 이하일 때만 파이 차트를 허용하는 조건을 함수에 추가해 보자

힌트: `if len(labels) <= 5:` 로 분기 처리

M2

누군가 ylim을 다시 임의로 덮어쓰려 하면 경고가 뜨도록, 함수 마지막에 `ax.get_ylim()[0]==0`을 `assert`로 확인해보자

힌트: `assert ax.get_ylim()[0] == 0, "0축이 아닙니다!"`

M3

이 함수를 `email_subject_test.csv`의 오픈을 비교에도 적용해 보자

힌트: `honest_bar(rates.index, rates.values, source="email_subject_test.csv")`

다 끝난 사람은: `source` 인자가 비어 있으면 경고를 출력하도록 함수를 보완해 볼 것

16개 지점, 착시 버전과 정직한 버전

조별(2~3명)로 12분. automation_pilot_savings.csv — 16개 지점의 자동화 파일럿 월 절감액(만원)입니다.

① 착시 버전 만들기

- 13강 체크리스트의 기법 중 하나(축 자르기·순서 조작·부적절한 차트) 이상을 골라 적용
- 실제보다 지점 간 차이가 훨씬 커 보이게 만들 것
- 어떤 기법을 왜 골랐는지 한 줄로 메모

② 정직한 버전으로 고치기

- 같은 데이터로, 13강 체크리스트를 모두 지켜서 다시 그리기 (가능하면 PART4의 honest_bar() 재사용)
- 두 버전의 차이를 lie factor 등 숫자로 설명



발표 형식: “우리는 ○○ 기법으로 착시를 만들었고, 정직한 버전과 비교하면 ○○배 차이가 났습니다.”

발표가 끝난 뒤에 함께 확인



model_answer.py

```
ap = pd.read_csv("automation_pilot_savings.csv")
a = ap[ap.site=="A"].monthly_saving_manwon.iloc[0]
b = ap[ap.site=="B"].monthly_saving_manwon.iloc[0]

actual = a / b
visual = (a-150) / (b-150) # 축 150부터
print(actual, visual, visual/actual)
```

실행 결과

A=253, B=223
실제비율=1.14, 시각비율=1.41
lie factor=1.24

정직한 버전

0에서 시작하는 축으로 16개 지점을 절감액 순으로 정렬해 막대 차트로 그리면, 진짜 차이 (83~327만원)가 그대로 보인다.



축을 150부터 자르기만 해도 lie factor 1.24 — 큰 격차처럼 안 보여도, 작은 조작이 쌓이면 인상이 크게 바뀐다.

오늘 만든 차트, 6가지 기준으로 다시 보기

체크리스트	오늘 실습에서 확인한 것
y축이 0에서 시작하는가	PART1 — lie factor 3.98배 (base=40)
기간·범위가 전체를 대표하는가	PART2 — cherry-picked 6개월 vs 전체 24개월
차트 유형이 목적에 맞는가	PART3 — 파이차트 오용(합≠100%)
스케일이 다른 지표를 억지로 겹치지 않았는가	PART3 — 이중축 남용 → rate_per_100 정규화
체크리스트가 코드에 강제되어 있는가	PART4 — honest_bar() 함수로 자동화
불필요한 장식이 없는가	PART5 — 착시 버전과 정직한 버전 직접 비교



오늘 하루 동안 이 체크리스트를 네 번 넘게 직접 적용해봤다 — 이제 여러분의 습관이 될 차례다.

오늘의 핵심 정리

- ✓ 축 조작: 같은 숫자라도 0축 대신 절단축을 쓰면 lie factor 3.98배까지 부풀려진다
- ✓ 순서·기간 조작: 정렬 기준과 cherry-picked 기간이 없는 추세를 만들어낸다
- ✓ 차트 유형 오용: 파이 차트는 부분-전체가 아닌 데이터에, 이중축은 서로 다른 스케일에 위험하다
- ✓ 함수화: honest_bar() 하나로 체크리스트를 코드가 강제하게 만들 수 있다
- ✓ 해법은 하나 — 0 기준선, 전체 맥락, 정규화, 그리고 체크리스트를 습관(과 코드)으로 만드는 것



과제 (다음 시간까지)

뉴스·SNS에서 본 그래프 하나를 가져와, 13강 체크리스트로 점검하고 정직한 버전(가능하면 honest_bar()로)으로 다시 그린 노트북을 제출하세요.